

(Raft) In Search of an Understandable Consensus Algorithm by D. Ongaro & J. Ousterhout

CSCI 585 lecture by Shahram Ghandeharizadeh



- Motivation for Consensus Protocols
- Divide-and-conquer: Separation of concerns
 - Leader election
 - Log replication
 - Safety
- Elasticity



- Replicated state machines are deterministic. Each computes the same state and the same sequence of output.
 - Each server stores a log containing a series of commands executed in order.
- Why? Enhance availability of data.
 - State machines on a collection of servers compute identical copies of the same state and may continue operating as long as the majority of the servers are available. E.g., ZooKeeper, Chubby.
- Challenge: How to keep replicated log consistent across the servers? This is the job of the consensus algorithm.
 - Paxos
 - Claims Paxos is too difficult to understand → Raft

Replicated State Machines

- Replicated state machines are deterministic. Each computes the same state and the same sequence of output.
 - Each server stores a log containing a series of commands executed in order.

- Why? Enhanc

- State machine of the same s majority of th

- Challenge: Hc servers? This

- Paxos
 - Claims Paxos

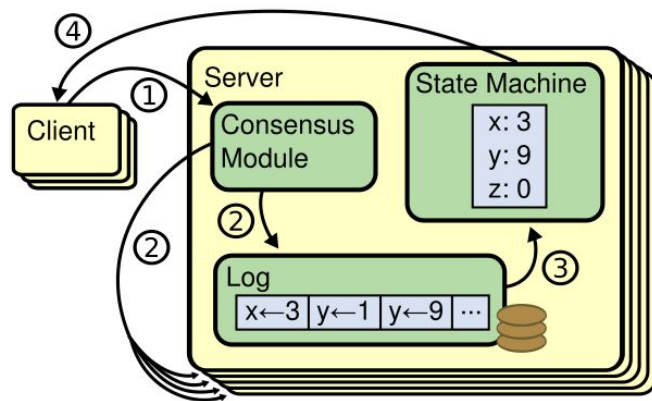


Figure 1: Replicated state machine architecture. The consensus algorithm manages a replicated log containing state machine commands from clients. The state machines process identical sequences of commands from the logs, so they produce the same outputs.

ute identical copies as long as the leader, Chubby.

istent across the orithm.

raft



- Assume a cluster with a fixed number of replicated servers.
- Section 6 of the paper removes this assumption.
- Consensus:
 - Raft elects a leader responsible for managing the replicated log.
 - Leader accepts log entries from clients, replicates them on other servers, and once acknowledged by a majority of the servers tells servers that it is safe to apply log entries to their state machine AND informs the client that the log entry has committed.
- Log records (data) flows from the leader to other servers.

Separation of concern:

- Leader election
- Log replication
- Safety



- Assume a cluster with 5 servers.
- Majority, 3 servers, are required to process reads and writes.
 - The system tolerates 2 failures.
- Each server has a state: leader, follower, or candidate.
 - Normal operation: 1 leader and 4 followers.
 - Raft allows at most one leader in a given term.
 - Leader processes all client requests. If a client issues its request to a follower, it is redirected to the leader.
 - Followers are passive, responding to requests from leaders and candidates.
 - Candidate is used to elect a new leader.
- Divide time into **terms** of arbitrary length. Terms are monotonically increasing. Each term begins with an election where one or more candidates attempt to become a leader. If a candidate wins the election, then it serves as leader for the rest of the term. (Think of the term as a logical clock.)

- Each server stores a current term number. Whenever servers communicate, they exchange their current term.
- If a leader discovers that its term is out of date, it immediately changes its state to follower. The same is true for a candidate.
- If a server receives a request with a stale term number, it rejects the request.
- Two types of RPC messages only:
 - a. RequestVote: Initiated by candidates during election.
 - b. AppendEntries: Initiated by leaders to replicate log entries in the form of heartbeat.
- Servers issue RPC in parallel for best performance. And, retry RPC if they do not receive a response in a timely manner.

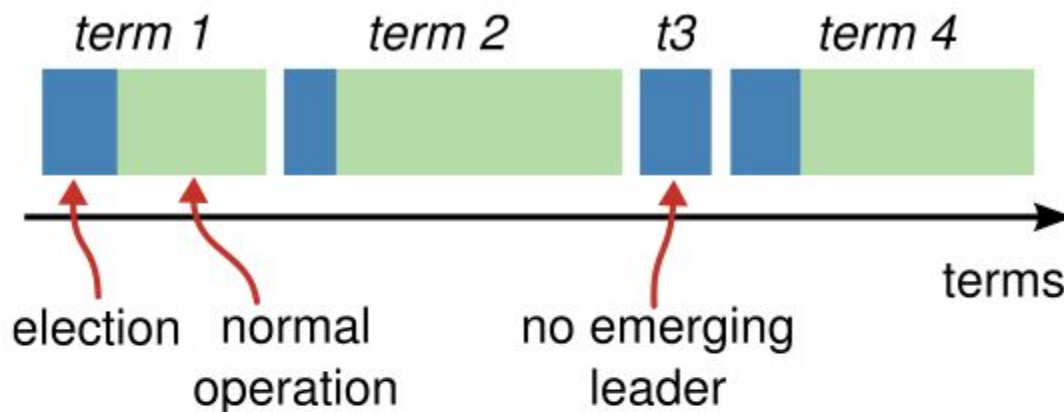


Figure 5: Time is divided into terms, and each term begins with an election. After a successful election, a single leader manages the cluster until the end of the term. Some elections fail, in which case the term ends without choosing a leader. The transitions between terms may be observed at different times on different servers.

Raft: What do You Know?

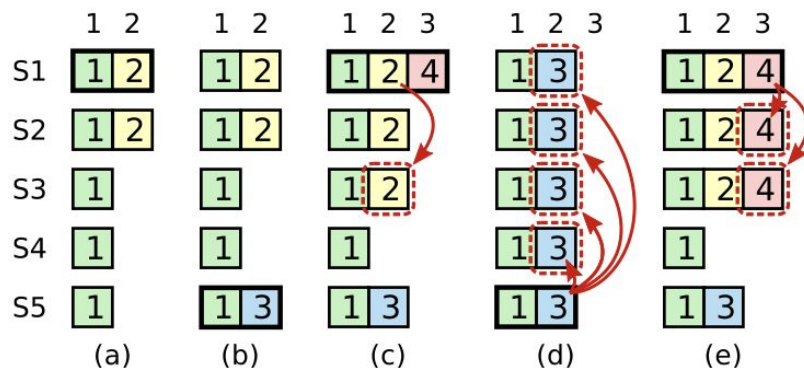


Figure 8: A time sequence showing why a leader cannot determine commitment using log entries from older terms. In (a) S1 is leader and partially replicates the log entry at index 2. In (b) S1 crashes; S5 is elected leader for term 3 with votes from S3, S4, and itself, and accepts a different entry at log index 2. In (c) S5 crashes; S1 restarts, is elected leader, and continues replication. At this point, the log entry from term 2 has been replicated on a majority of the servers, but it is not committed. If S1 crashes as in (d), S5 could be elected leader (with votes from S2, S3, and S4) and overwrite the entry with its own entry from term 3. However, if S1 replicates an entry from its current term on a majority of the servers before crashing, as in (e), then this entry is committed (S5 cannot win an election). At this point all preceding entries in the log are committed as well.

- Leaders MUST send periodic heartbeats (AppendEntries with no log entries) to all followers.
- If a follower's timer expires (***election timeout***) then it assumes the leader is dead and begins an election to select a new leader. To begin an election:
 - a. Follower increments its current term and changes its state to candidate.
 - b. It votes for itself, i.e., starts with number of votes = 1.
 - c. Issues RequestVote RPC in parallel to each server in the cluster.
 - d. Three things may happen: it wins the election, another server establishes itself as a leader, a period of time goes by with no winner.

- Leaders MUST send periodic heartbeats (AppendEntries with
 - A candidate wins if it receives votes from 2 other servers, i.e., majority including itself.
 - Each server votes for at most one candidate in a given term, breaks ties using first-come-first-serve (FCFS) basis.
 - Once the candidate wins an election, it becomes leader and starts to send heartbeat messages to all other servers to establish its authority. It starts to process client requests.
- c. Issues RequestVote RPC in parallel to each server in the cluster.
- d. Three things may happen: it **wins the election**, another server establishes itself as a leader, a period of time goes by with no winner.

- If a candidate receives an AppendEntries RPC from a server claiming to be leader:
 - A candidate may receive an AppendEntries from another server claiming to be leader.
 - If the leader's term is at least as large as the candidate's current term then the candidate recognizes the leader as legitimate and returns to the follower state.
 - If the leader's term is smaller then the candidate rejects the RPC and continues in candidate state.
- If a candidate receives a RequestVote RPC from a server:
 - a. The candidate recognizes the server as a follower and votes for the candidate.
 - b. It votes for itself, i.e., starts with number of votes = 1.
 - c. Issues RequestVote RPC in parallel to each server in the cluster.
 - d. Three things may happen: it wins the election, **another server establishes itself as a leader**, a period of time goes by with no winner.

- Split vote: candidate neither wins nor loses the election.
- Each candidate uses randomized timeout (150-300 ms) and start a new election by incrementing its term and initiating another round of RequestVote RPC.

leader. To begin an election:

- a. Follower increments its current term and changes its state to candidate.
- b. It votes for itself, i.e., starts with number of votes = 1.
- c. Issues RequestVote RPC in parallel to each server in the cluster.
- d. Three things may happen: it wins the election, another server establishes itself as a leader, **a period of time goes by with no winner.**

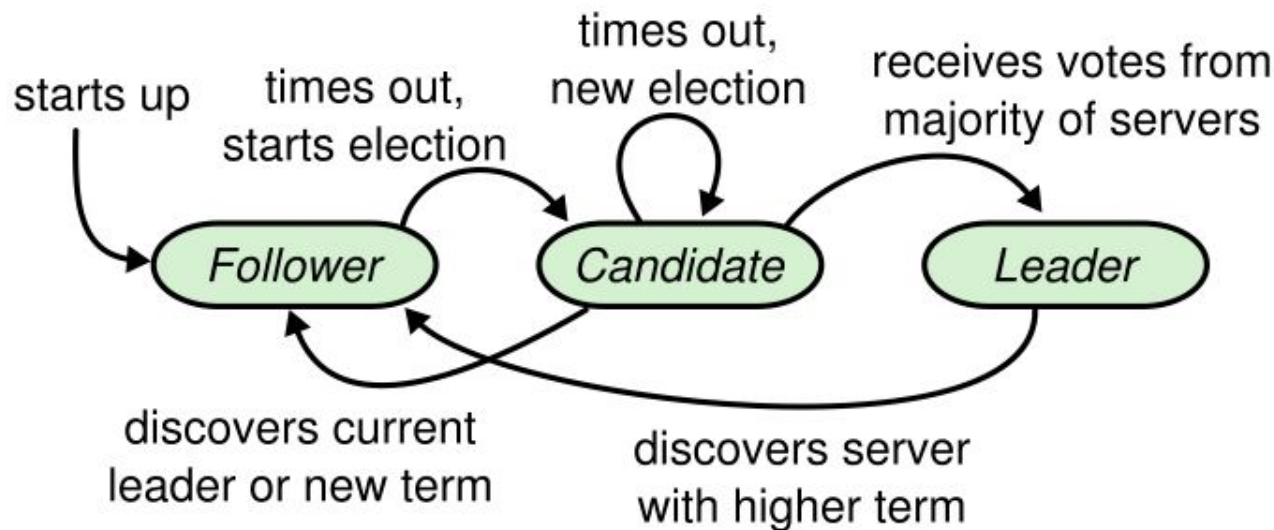


Figure 4: Server states. Followers only respond to requests from other servers. If a follower receives no communication, it becomes a candidate and initiates an election. A candidate that receives votes from a majority of the full cluster becomes the new leader. Leaders typically operate until they fail.



- A client request contains a command to be executed by the replicated state machine.
- Leader appends the command to its log as a new entry AND Issues AppendEntries RPCS in parallel to each of the other servers to replicate the entry.
- When the entry has been replicated across the majority (2 other servers), it is called committed. The leader applies the entry to its state machine and returns the result to the client.
- Leader retries AppendEntries indefinitely until all followers eventually store all log entries.



A client request contains a command to be executed by the the new entries. If the follower does not find an entry in its log with the same index and term, then it refuses the new entries. The consistency check acts as an induction servers to replicate the entry.

- When the entry has been replicated across the majority (2 other servers), it is called committed. The leader applies the entry to its state machine and returns the result to the client.
- Leader retries AppendEntries indefinitely until all followers eventually store all log entries.

Raft: Log Replication



A client request contains a command to be executed by the the new entries. If the follower does not find an entry in its log with the same index and term, then it refuses the new entries. The consistency check acts as an induction servers to replicate the entry.

As a result, whenever AppendEntries returns successfully, the leader knows that the follower's log is identical to its own log up through the new entries.
eventually store all log entries.

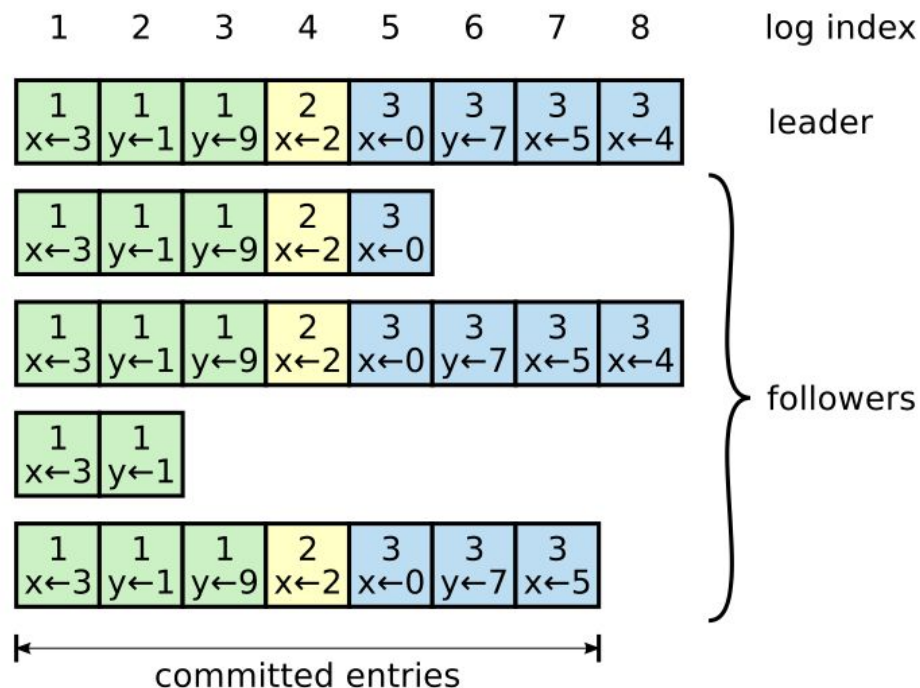
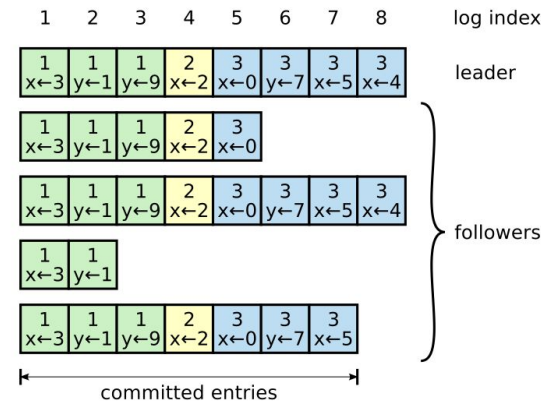


Figure 6: Logs are composed of entries, which are numbered sequentially. Each entry contains the term in which it was created (the number in each box) and a command for the state machine. An entry is considered *committed* if it is safe for that entry to be applied to state machines.

Raft: Logs

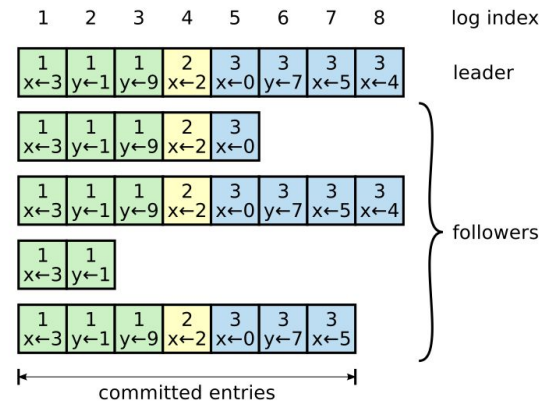
- Leader maintains the highest index it knows to be committed. What is it in this example?
- Leader includes the index in its AppendEntries RPC (heartbeat) messages.
- Once a follower learns that a long entry is committed, it applies the entry to its local state machine (in log order).



- If two entries in different logs have the same index and term, then they store the same command.
- If two entries in different logs have the same index and term, then the logs are identical in all preceding entries.

Raft: Normal Operation

- Logs of the leader and followers stay consistent.
- The AppendEntries consistency check never fails.
- Arbitrary crashes may leave the logs of different servers inconsistent with one another.



Raft: Follower with Missing Entries

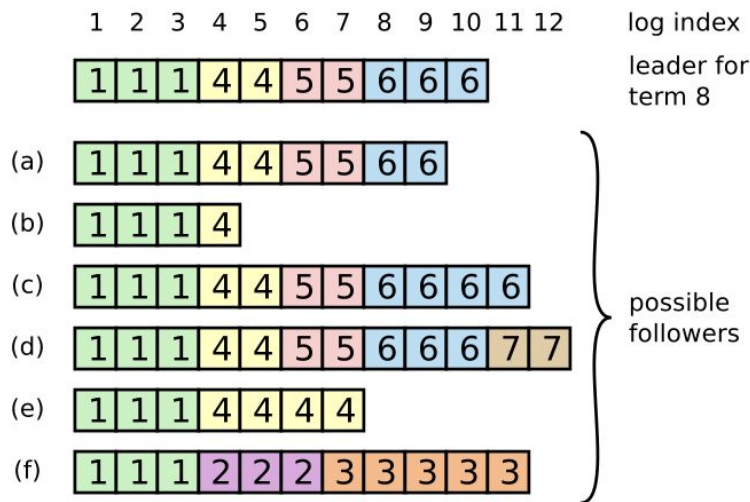


Figure 7: When the leader at the top comes to power, it is possible that any of scenarios (a–f) could occur in follower logs. Each box represents one log entry; the number in the box is its term. A follower may be missing entries (a–b), may have extra uncommitted entries (c–d), or both (e–f). For example, scenario (f) could occur if that server was the leader for term 2, added several entries to its log, then crashed before committing any of them; it restarted quickly, became leader for term 3, and added a few more entries to its log; before any of the entries in either term 2 or term 3 were committed, the server crashed again and remained down for several terms.

Raft: Follower with Missing Entries

RPCs. The leader maintains a *nextIndex* for each follower, which is the index of the next log entry the leader will send to that follower. When a leader first comes to power, it initializes all *nextIndex* values to the index just after the last one in its log (11 in Figure 7). If a follower's log is inconsistent with the leader's, the AppendEntries consistency check will fail in the next AppendEntries RPC. After a rejection, the leader decrements *nextIndex* and retries the AppendEntries RPC. Eventually *nextIndex* will reach a point where the leader and follower logs match. When this happens, AppendEntries will succeed, which removes any conflicting entries in the follower's log and appends entries from the leader's log (if any). Once AppendEntries succeeds, the follower's log is consistent with the leader's, and it will remain that way for the rest of the term.

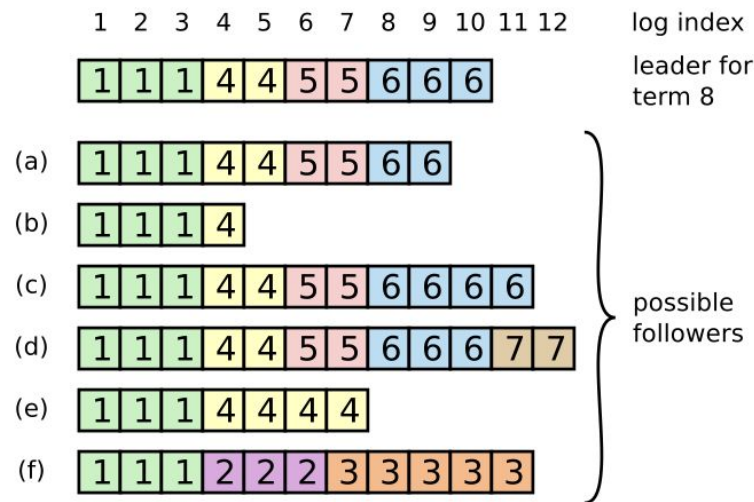


Figure 7: When the leader at the top comes to power, it is possible that any of scenarios (a–f) could occur in follower logs. Each box represents one log entry; the number in the box is its term. A follower may be missing entries (a–b), may have extra uncommitted entries (c–d), or both (e–f). For example, scenario (f) could occur if that server was the leader for term 2, added several entries to its log, then crashed before committing any of them; it restarted quickly, became leader for term 3, and added a few more entries to its log; before any of the entries in either term 2 or term 3 were committed, the server crashed again and remained down for several terms.

Raft: Follower with Missing Entries

RPCs. The leader maintains a *nextIndex* for each follower.

1 2 3 4 5 6 7 8 9 10 11 12 log index
leader for

With this mechanism, a leader does not need to take any special actions to restore log consistency when it comes to power. It just begins normal operation, and the logs automatically converge in response to failures of the Append-Entries consistency check. A leader never overwrites or deletes entries in its own log (the Leader Append-Only Property in Figure 3).

and it will remain that way for the rest of the term.

for term 2, added several entries to its log, then crashed before committing any of them; it restarted quickly, became leader for term 3, and added a few more entries to its log; before any of the entries in either term 2 or term 3 were committed, the server crashed again and remained down for several terms.

This is not safe.

same order. For example, a follower might be unavailable while the leader commits several log entries, then it could be elected leader and overwrite these entries with new ones; as a result, different state machines might execute different command sequences.

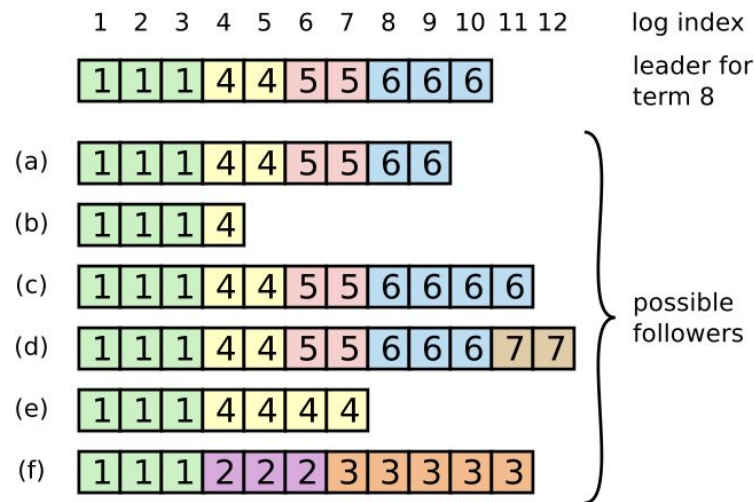


Figure 7: When the leader at the top comes to power, it is possible that any of scenarios (a–f) could occur in follower logs. Each box represents one log entry; the number in the box is its term. A follower may be missing entries (a–b), may have extra uncommitted entries (c–d), or both (e–f). For example, scenario (f) could occur if that server was the leader for term 2, added several entries to its log, then crashed before committing any of them; it restarted quickly, became leader for term 3, and added a few more entries to its log; before any of the entries in either term 2 or term 3 were committed, the server crashed again and remained down for several terms.



- Add a restriction on which servers may be elected leader.
- Raft's voting process prevents a candidate from winning an election unless its log contains all committed entries.

RequestVote RPC implements this restriction: the RPC includes information about the candidate's log, and the voter denies its vote if its own log is more up-to-date than that of the candidate.

Raft: Counting Replicas

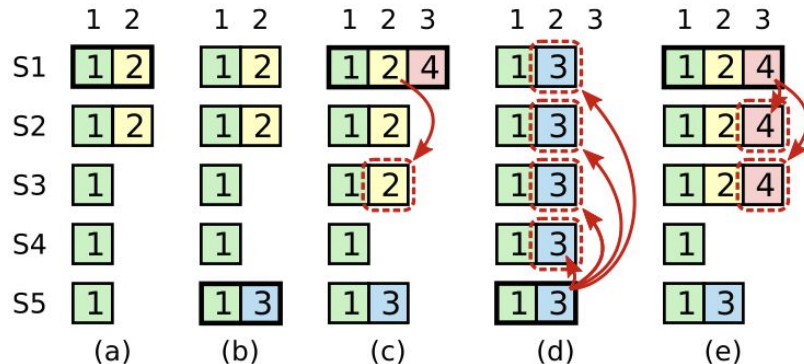


Figure 8: A time sequence showing why a leader cannot determine commitment using log entries from older terms. In (a) S1 is leader and partially replicates the log entry at index 2. In (b) S1 crashes; S5 is elected leader for term 3 with votes from S3, S4, and itself, and accepts a different entry at log index 2. In (c) S5 crashes; S1 restarts, is elected leader, and continues replication. At this point, the log entry from term 2 has been replicated on a majority of the servers, but it is not committed. If S1 crashes as in (d), S5 could be elected leader (with votes from S2, S3, and S4) and overwrite the entry with its own entry from term 3. However, if S1 replicates an entry from its current term on a majority of the servers before crashing, as in (e), then this entry is committed (S5 cannot win an election). At this point all preceding entries in the log are committed as well.

Election Safety: at most one leader can be elected in a given term. §5.2

Leader Append-Only: a leader never overwrites or deletes entries in its log; it only appends new entries. §5.3

Log Matching: if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index. §5.3

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms. §5.4

State Machine Safety: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index. §5.4.3

Figure 3: Raft guarantees that each of these properties is true at all times. The section numbers indicate where each property is discussed.

Election Safety: at most one leader can be elected in a given term. §5.2

Leader Append-Only: a leader never overwrites or deletes entries in its log; it only appends new entries. §5.3

Log Matching: if two logs contain an entry with the same index and term, then the logs are identical in all entries up through the given index. §5.3

Leader Completeness: if a log entry is committed in a given term, then that entry will be present in the logs of the leaders for all higher-numbered terms. §5.4

State Machine Safety: if a server has applied a log entry at a given index to its state machine, no other server will ever apply a different log entry for the same index. §5.4.3

Figure 3: Raft guarantees that each of these properties is true at all times. The section numbers indicate where each property is discussed.

Raft: Follower/Candidate Failures



- Leader's AppendEntries RPCs will fail. Leader will retry indefinitely.
- A candidate's RequestVote will fail. Candidate will retry indefinitely.
- If a server crashes after completing an RPC but before responding, then it will receive the same RPC again after it restarts.
 - a. Raft assumes its RPCs are idempotent.

- The following timing requirement enables Raft to make progress:

$$\textit{broadcastTime} \ll \textit{electionTimeout} \ll \textit{MTBF}$$

- BroadcastTime: Avg time for a server to send RPCs in parallel to every server in the cluster and receive their responses.
 - ElectionTimeout: Inter-arrival time between periodic heartbeat (AppendEntries RPC) messages by the leader.
 - When the leader fails, the system will be unavailable for roughly the ElectionTimeout.
 - MTBF: Avg time between failures for a single server.
-
- BroadcastTime should be an order of magnitude faster than ElectionTimeout. This prevents followers from starting elections.
 - ElectionTimeout should be a few orders of magnitude faster than MTBF to enable RAFT to make steady progress.

Raft: Typical Times

- The following timing requirement enables Raft to make progress:

$$\textit{broadcastTime} \ll \textit{electionTimeout} \ll \textit{MTBF}$$

- BroadcastTime and MTBF are properties of the system. ElectionTimeout is chosen by the system designer.
- Raft's AppendEntries requires the recipient to persistent information to stable storage. This means BroadcastTime may range from 0.5 to 20 ms.
- ElectionTimeout is between 10 ms to 500 ms.
- MTBF is typically several months or more.

Raft: Time to Elect a New Leader

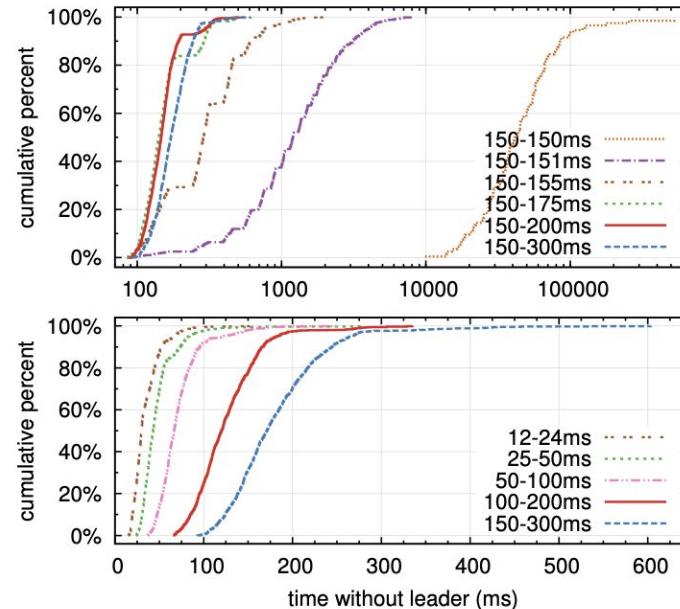


Figure 14: The time to detect and replace a crashed leader. The top graph varies the amount of randomness in election timeouts, and the bottom graph scales the minimum election timeout. Each line represents 1000 trials (except for 100 trials for “150–150ms”) and corresponds to a particular choice of election timeouts; for example, “150–155ms” means that election timeouts were chosen randomly and uniformly between 150ms and 155ms. The measurements were taken on a cluster of five servers with a broadcast time of roughly 15ms. Results for a cluster of nine servers are similar.

Raft Elasticity

Presented on Monday, Nov 17



Raft: Elasticity Challenge

- Configuration change to replace servers when they fail or to change the degree of replication.
- Simple solution: Take the cluster off-line, update configuration files, and then restart the cluster.
 - Disadvantage: Data becomes unavailable during the changeover. Manual steps may risk operator error.
- On-line configuration change:
 - Requirement: No instance of time with two leaders for the same term.
 - Switch from the old configuration to the new configuration is unsafe. Figure 10 shows this in the form of 2 different leaders for the same term.

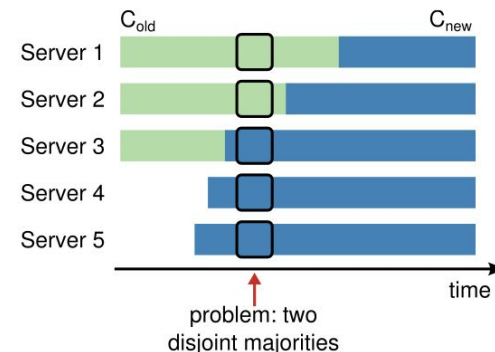


Figure 10: Switching directly from one configuration to another is unsafe because different servers will switch at different times. In this example, the cluster grows from three servers to five. Unfortunately, there is a point in time where two different leaders can be elected for the same term, one with a majority of the old configuration (C_{old}) and another with a majority of the new configuration (C_{new}).

Raft: Elasticity Solution

- A 2-phase approach:
 - Phase 1: Switch to a transitional configuration termed joint consensus.
 - Phase 2: Commit the joint consensus and transition to the new configuration.
- Joint consensus allows individual servers to transition from old to new at different times without compromising safety and while serving requests. It combines both the old and new configurations:
 - Log entries are replicated to all servers in both configurations.
 - Any server from either configuration may serve as leader.
 - Agreement for election and entry commitment requires separate majorities from **both** the old and new configurations.

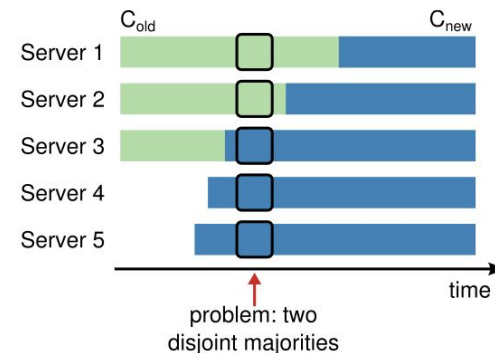


Figure 10: Switching directly from one configuration to another is unsafe because different servers will switch at different times. In this example, the cluster grows from three servers to five. Unfortunately, there is a point in time where two different leaders can be elected for the same term, one with a majority of the old configuration (C_{old}) and another with a majority of the new configuration (C_{new}).

Raft: Cluster Configurations

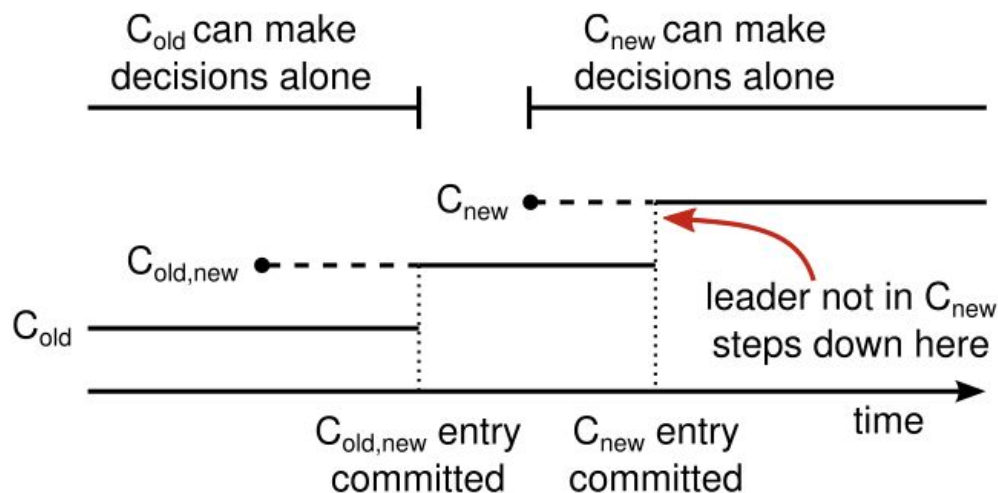
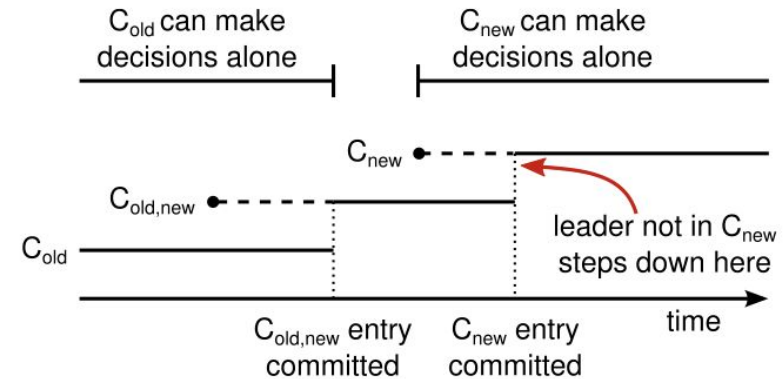


Figure 11: Timeline for a configuration change. Dashed lines show configuration entries that have been created but not committed, and solid lines show the latest committed configuration entry. The leader first creates the $C_{old,new}$ configuration entry in its log and commits it to $C_{old,new}$ (a majority of C_{old} and a majority of C_{new}). Then it creates the C_{new} entry and commits it to a majority of C_{new} . There is no point in time in which C_{old} and C_{new} can both make decisions independently.

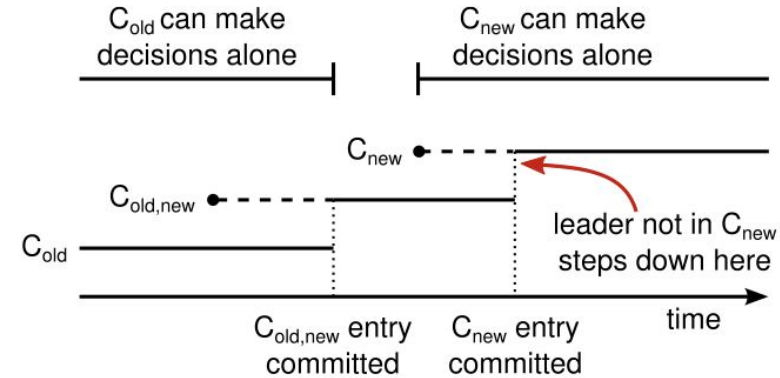
Raft: C_{old} to C_{new}

trates the configuration change process. When the leader receives a request to change the configuration from C_{old} to C_{new} , it stores the configuration for joint consensus ($C_{old,new}$ in the figure) as a log entry and replicates that entry using the mechanisms described previously. Once a given server adds the new configuration entry to its log, it uses that configuration for all future decisions (a server always uses the latest configuration in its log, regardless of whether the entry is committed). This means that the leader will use the rules of $C_{old,new}$ to determine when the log entry for $C_{old,new}$ is committed. If the leader crashes, a new leader may be chosen under either C_{old} or $C_{old,new}$, depending on whether the winning candidate has received $C_{old,new}$. In any case, C_{new} cannot make unilateral decisions during this period.



Raft: $C_{old,new}$ to C_{new}

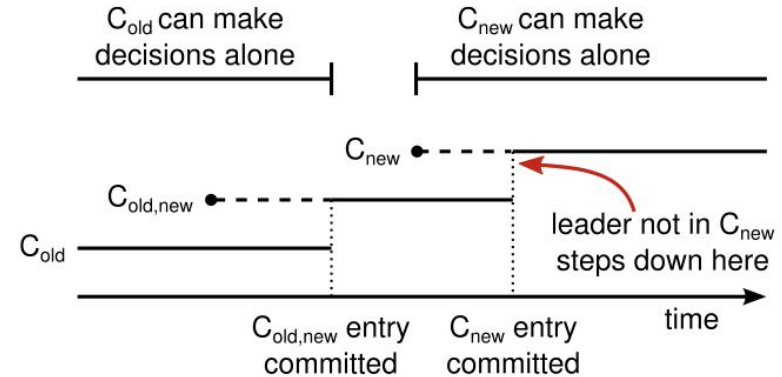
Once $C_{old,new}$ has been committed, neither C_{old} nor C_{new} can make decisions without approval of the other, and the Leader Completeness Property ensures that only servers with the $C_{old,new}$ log entry can be elected as leader. It is now safe for the leader to create a log entry describing C_{new} and replicate it to the cluster. Again, this configuration will take effect on each server as soon as it is seen. When the new configuration has been committed under the rules of C_{new} , the old configuration is irrelevant and servers not in the new configuration can be shut down. As



There is no time when C_{old} and C_{new} are both able to make decisions. This guarantees safety.

Raft: Non-Voting Members

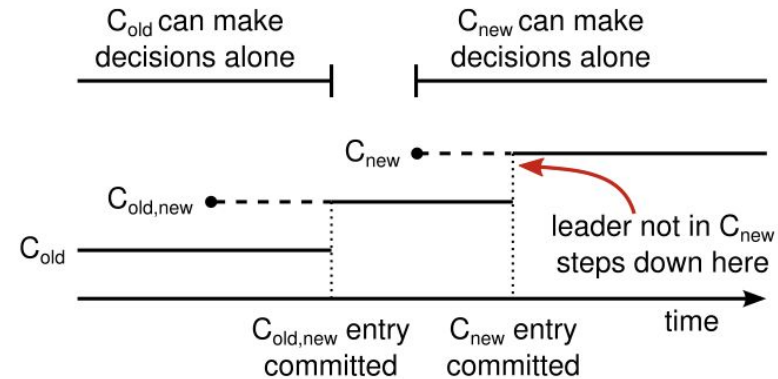
Raft introduces an additional phase before the configuration change, in which the new servers join the cluster as non-voting members (the leader replicates log entries to them, but they are not considered for majorities). Once the new servers have caught up with the rest of the cluster, the reconfiguration can proceed as described above.



Why? New servers may not initially store log entries. Adding such servers to a cluster may require a while for them to catch up. During this time, when the new servers are the majority, it may not be possible to commit new log entries.

Raft: What if we want to remove the leader?

The second issue is that the cluster leader may not be part of the new configuration. In this case, the leader steps down (returns to follower state) once it has committed the C_{new} log entry. This means that there will be a period of time (while it is committing C_{new}) when the leader is managing a cluster that does not include itself; it replicates log entries but does not count itself in majorities. The leader transition occurs when C_{new} is committed because this is the first point when the new configuration can operate independently (it will always be possible to choose a leader from C_{new}). Before this point, it may be the case that only a server from C_{old} can be elected leader.



Raft: Removed Servers May Disrupt the Cluster

The third issue is that removed servers (those not in C_{new}) can disrupt the cluster. These servers will not receive heartbeats, so they will time out and start new elections. They will then send RequestVote RPCs with new term numbers, and this will cause the current leader to revert to follower state. A new leader will eventually be elected, but the removed servers will time out again and the process will repeat, resulting in poor availability.

To prevent this problem, servers disregard RequestVote RPCs when they believe a current leader exists. Specifically, if a server receives a RequestVote RPC within the minimum election timeout of hearing from a current leader, it does not update its term or grant its vote.

This does not affect normal elections, where each server waits at least a minimum election timeout before starting an election. However, it helps avoid disruptions from removed servers: if a leader is able to get heartbeats to its cluster, then it will not be deposed by larger term numbers.

